

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Práctica 4:

FilmStars

PONDERACIÓN: 3 pts

Índice

Descripción del problema	3
Alcance del sistema	4
1. Módulo de Autenticación y Sesión	4
2. Selección de Ubicación y Funciones	4
3. Visualización de películas por categoría	4
4. Selección de Asientos en Tiempo real	4
5. Flujo de Compra de Boleto	5
6. Módulo Administrativo (NUEVO ELEMENTO)	5
7. Catálogo Optimizado y Paginación (Modulo de Usuario - NUEVO)	5
8. Restricciones de Acceso End-to-End	5
Requisitos de Automatización (NUEVO/ACTUALIZACIÓN)	6
1. Fase Test (Fase CI):	6
2. Fase de Build & Push (Registry)	6
3. Fase de Deploy (Fase CD en VM)	6
Requisitos y restricciones	7
Requisitos Técnicos	7
1. Principios SOLID (Calidad de código)	7
Entregables Requeridos	7
1. Código Fuente e Implementación	7
2. Requerimientos funcionales	8
3. Modelado de Casos de Uso	8
4. Modelo 4+1 Vistas de Kruchten	8
5. Diagrama de Arquitectura de Alto Nivel/Bloques (Cómo lo conozcan)	8
6. Diagrama de Flujo del Pipeline CI/CD:	8
7. Documentación SOLID:	8
8. Justificación y toma de decisiones:	9
Herramientas permitidas	9
Cronograma	10
Consideraciones	10

Descripción del problema

Una importante cadena de cines desea modernizar su plataforma digital para permitir la venta de boletos en línea. El sistema debe de ser capaz de gestionar picos altos de demanda de manera eficiente garantizando que no ocurran condiciones de carrera al seleccionar asientos de forma simultánea, y evitando la pérdida de transacciones financieras ante caídas del sistema.

Para cumplir con estos atributos de calidad, el departamento de ingeniería ha determinado que el sistema se construirá bajo un enfoque de Arquitectura Orientada a Servicios (SOA). La comunicación crítica entre servicios debe gestionarse obligatoriamente de forma asíncrona, utilizando un middleware de mensajería basado en colas.

(Practica 2)

En la práctica anterior, se diseñó la arquitectura base para el sistema de reserva y venta de boletos de cine bajo un enfoque Orientado a Servicios (SOA). Para esta segunda iteración, el objetivo principal es trasladar el diseño a un sistema de software completamente funcional, elevando los estándares de calidad en la construcción del código e implementando el esquema de seguridad para el control de acceso.

El desarrollo de esta fase exige la aplicación rigurosa de los Principios SOLID de diseño orientado a objetos. Esto garantizará que los servicios (como la cartelera, usuarios o pagos) sean tolerantes al cambio, fáciles de mantener y probar de forma aislada. Asimismo, se debe asegurar la frontera del sistema implementando autenticación basada en claims mediante JWT (JSON Web Tokens).

(Práctica 3)

En las iteraciones anteriores, se diseñó la arquitectura base SOA y se construyó el flujo central de cara al cliente (comprar boletos y autenticación) utilizando contenedores Docker. No obstante, una plataforma comercial requiere de un gobierno operativo robusto y de un flujo de entrega de código que garantice la estabilidad del sistema ante cambios continuos en producción.

(Práctica 4)

A medida que el sistema de reserva y venta de boletos de cine madura, el volumen de información y la complejidad de los despliegues escalan. Cargar contenidos uno a uno a través de interfaces gráficas tradicionales resulta inviable cuando las distribuidoras liberan catálogos masivos. De igual manera, renderizar cientos de películas simultáneamente en el cliente degrada la latencia y la experiencia de usuario, haciendo mandatoria la optimización del tráfico de red mediante estrategias de segmentación de datos.

En el frente de operaciones (DevOps), los entornos productivos exigen que los servidores no compilen código ni construyan imágenes en el momento (*no builds in production*), ya que esto consume recursos críticos y genera inconsistencias entre versiones.

Para mitigar estas problemáticas, en esta cuarta práctica se exige:

1. **Eficiencia en Datos:** Implementar un motor de ingesta masiva de información y optimizar las consultas de cartelera mediante paginación del lado del servidor.
2. **Infraestructura Inmutable (DevOps):** Diseñar un pipeline de CI/CD que centralice la construcción de artefactos en un registro público obligatorio (Docker Hub) y automatice el despliegue en Máquinas Virtuales (VM) utilizando únicamente el mecanismo de descarga (*pull*) de imágenes precompiladas.

Alcance del sistema

El sistema final entregado debe fusionar las capacidades de la práctica anterior con los nuevos componentes de seguridad:

1. Módulo de Autenticación y Sesión

- **Registro de usuarios:** Formulario con validaciones en el frontend y persistencia segura (contraseñas encriptadas en la base de datos).
- **Inicio de Sesión:** Validación de credenciales en el servicio de usuarios.
- **Emisión y Gestión de JWT:** Tras un login exitoso, el backend debe firmar y retornar un token JWT que almacene los claims esenciales del usuario (ej. ID, Nombre, Rol). El frontend debe capturar y enviar este token en la cabecera HTTP (*Authorization: Bearer <token>*) para acceder a las rutas protegidas (como el flujo de compra y selección de asientos).

2. Selección de Ubicación y Funciones

- El usuario debe de poder seleccionar su ciudad para visualizar dinámicamente los cines y los horarios/funciones disponibles en dicha localidad.

3. Visualización de películas por categoría

- Clasificación y despliegue de la cartelera segmentada según el tipo de proyección: Estrenos, Pre-ventas, Re-Estrenos.

4. Selección de Asientos en Tiempo real

- Mapa interactivo de la sala para la función seleccionada. Se debe mapear visualmente el estado de cada asiento (Disponible, Ocupado, Seleccionado/Bloqueado temporalmente)

5. Flujo de Compra de Boleto

- Procesamiento del flujo desde que se confirma la selección de asientos, se envía la cola de mensajería para su validación, se procesa el pago simulado y se emite el boleto final.

6. Módulo Administrativo (NUEVO ELEMENTO)

El sistema debe incorporar un **Panel de Administración** web exclusivo para usuarios con rol de administrador (*RoI: Admin*), el cual debe permitir operaciones CRUD (*Crear, Leer, Actualizar, Borrar*) sobre las siguientes entidades:

- **Gestión de Cines y Salas:** Configuración de sedes físicas, número de salas y su respectivo mapa base de asientos.
- **Gestión de Horarios y Funciones:** Asignación de películas a salas específicas en horarios determinados, definiendo si la función corresponde a un *Estreno*, *Pre-venta* o *Re-estreno*.
- **Carga de Archivos CSV (NUEVO):** El Panel de Administración debe proveer una opción para cargar un archivo en formato *.csv* con múltiples registros de películas simultáneos.
- **Flexibilidad de entrada (“NUEVO”):** El administrador mantendrá la dualidad operativa; podrá seguir insertando películas de forma manual (uno a uno) o delegar el proceso al motor de carga masiva.

Nota: La estructura y columnas del archivo *.csv* (ej. título, sinopsis, duración, clasificación, tipo de función) quedan a libre discreción técnica del estudiante, pero deben estar normalizadas.

7. Catálogo Optimizado y Paginación (Modulo de Usuario - NUEVO)

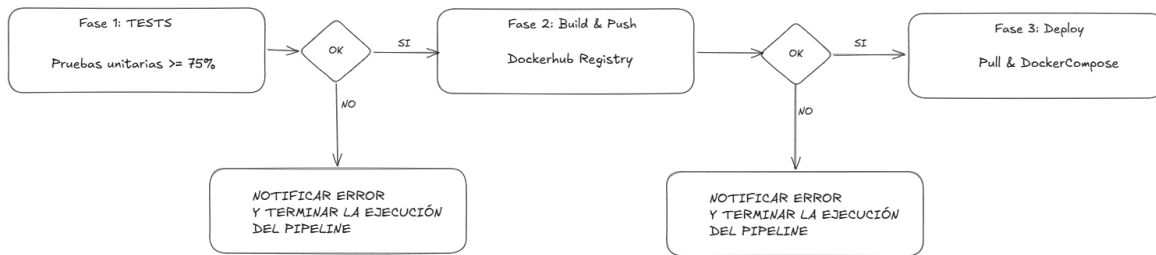
- **Paginación del Lado del Servidor:** La vista de la cartelera general de películas debe implementar paginación obligatoria, desplegando un **máximo de 10 películas por página**.
- **Consistencia en Filtros:** Los controles de paginación (Siguiente, Anterior, Número de Página) deben mantenerse completamente funcionales incluso cuando el usuario aplique filtros de búsqueda o segmentaciones por ciudad y tipo de función (*Estrenos*, *Preventas*, *Re-estrenos*). Queda estrictamente prohibido traer todo el catálogo del backend y paginar en el cliente.

8. Restricciones de Acceso End-to-End

- Tanto las rutas del Frontend como los endpoints del Backend SOA dedicados a la administración deben validar de forma estricta que el **JWT** de la sesión pertenezca a un administrador legítimo. Queda prohibido permitir que un usuario común interactúe con estas vistas o APIs.

Requisitos de Automatización (NUEVO/ACTUALIZACIÓN)

Al impactar o realizar un *push/merge* directo en la rama de **release**, el pipeline de CI/CD (GitHub Actions, GitLab CI/CD, etc.) debe ejecutar de forma obligatoria y secuencial las siguientes tres fases:



1. Fase Test (Fase CI):

Ejecución automática de las pruebas unitarias en el backend. El pipeline se cancelará si la cobertura de los endpoints utilizados cae por debajo del **75% de cobertura**.

2. Fase de Build & Push (Registry)

Compilación y construcción automatizada de las imágenes Docker (Frontend y servicios Backend). Las imágenes resultantes deben ser empujadas obligatoriamente a un registro público en **Docker Hub**. Cada imagen debe ser etiquetada con el tag **latest**.

3. Fase de Deploy (Fase CD en VM)

El pipeline se conectará de manera segura a la Máquina Virtual (VM) de producción y ejecutará un despliegue automatizado por medio de **Docker Compose**.

- Se debe manejar un deploy en AWS para cuando se accione un push en la rama **release**.
- Se debe manejar un deploy en una computadora de los integrantes del grupo cuando se accione un push en la rama **develop**.
 - **Restricción estricta:** El archivo `docker-compose.yml` en producción tiene prohibido compilar código local (`build: .`). El despliegue debe realizarse única y exclusivamente descargando las imágenes empaquetadas desde Docker Hub (`image: dockerhub_usuario/servicio:tag`).

Requisitos y restricciones

Cada dominio conceptual (Usuario, Películas/Cartelera, Reservas/Asientos, Pagos) debe de modelarse como servicio independiente con su propio almacenamiento o lógica de acceso a datos.

Los procesos donde la concurrencia sea crítica deben de desacoplarse mediante un bróker de mensajería. Las peticiones no deben bloquear el hilo principal de la aplicación, en su lugar, deben de entrar a cola y serán procesadas por consumidores independientes.

Esta práctica **NO** consiste únicamente en la entrega de endpoints o APIs en el backend. Se requiere obligatoriamente una **aplicación web funcional de extremo a extremo (End-to-End)**, lo que incluye una interfaz de usuario (Frontend) interactiva e integrada con los servicios mediante consumo de APIs

Requisitos Técnicos

1. Principios SOLID (Calidad de código)

Los estudiantes deberán demostrar de forma explícita en su código fuente la aplicación de los siguientes principios:

- **S - Single Responsibility Principle (Responsabilidad Única)**
Cada clase, controlador o servicio de backend debe tener una única razón para cambiar (ej. separar la lógica de negocio de la reserva de la lógica de persistencia de datos o del envío a la cola de mensajería).
- **O - Open/Closed Principle (Abierto/Cerrado):**
El software debe estar abierto para su extensión pero cerrado para su modificación (ej. el sistema de cálculo de precios según el tipo de función—estreno, preventa o re-estreno—debe permitir añadir nuevas reglas sin modificar las clases existentes).
- **L - Liskov Substitution Principle (Sustitución de Liskov):**
Las clases derivadas deben poder sustituir a sus clases base sin alterar el comportamiento del programa.
- **I - Interface Segregation Principle (Segregación de Interfaces):**
Los clientes no deben ser obligados a depender de interfaces o métodos que no utilizan.
- **D - Dependency Inversion Principle (Inversión de Dependencias):**
Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones (uso estricto de interfaces para desacoplar controladores de servicios y servicios de repositorios de datos).

Entregables Requeridos

1. Código Fuente e Implementación

- **Repositorio de Código (GitHub/GitLab):** Proyecto organizado de manera limpia con un archivo *.md* explicativo que incluya los pasos detallados para levantar tanto el Frontend como el Backend y el entorno con Docker Compose
- **Archivos de Configuración Docker:** Inclusión de todos los *Dockerfiles* y el script *docker-compose.yml* completamente funcional.
- **Scripts de CI/CD:** Archivo de configuración del pipeline.
- **Evidencia de Cobertura:** Captura de pantalla o logs del pipeline demostrando el cumplimiento del 75% de cobertura.
- **Aplicación End-to-End:** Frontend funcional integrado y consumiendo los servicios del backend a través de la red interna de Docker.
- Creación del Tag con la versión **V2.2.0**

2. Requerimientos funcionales

Actualización de los Requerimientos Funcionales (RF) para la carga masiva y paginación, junto con Requerimientos No Funcionales (RNF) enfocados en la inmutabilidad de imágenes y tiempos de respuesta de consultas paginadas.

3. Modelado de Casos de Uso

Inclusión del caso de uso expandido para la "Carga masiva de películas desde CSV" con su narrativa (flujos principales y de excepción) y su respectivo diagrama UML local.

4. Modelo 4+1 Vistas de Kruchten

Actualización completa de las 5 vistas para reflejar la integración de Docker Hub como repositorio de artefactos, el flujo inmutable hacia la VM y las nuevas capacidades de software.

5. Diagrama de Arquitectura de Alto Nivel/Bloques (Cómo lo conocen)

Modificación del esquema conceptual para integrar el API Gateway, la VM de producción, las capas de datos y el motor de CI/CD interactuando con Docker Hub.

6. Diagrama de Flujo del Pipeline CI/CD:

Modelado secuencial detallado que grafique las compuertas de decisión de las tres fases obligatorias (*Tests >= 75%*, *Build & Push a Docker Hub con Tags*, y *Deploy por Pull en Docker Compose*).

7. Documentación SOLID:

Actualización de calidad de diseño. Se debe documentar explícitamente qué principios SOLID se aplicaron o expandieron para dar soporte a la lectura del archivo CSV y a los algoritmos de paginación del catálogo

8. Justificación y toma de decisiones:

Ampliación de la justificación técnica incorporando las secciones explicativas para:

- Registry Dockerhub
- GithubActions para CI/CD
- Herramienta o Herramientas para pruebas unitarias
- Nube utilizada
- Sistemas de la nube utilizados
- Formato para Carga Masiva

Herramientas permitidas

Tipo	Categoría	Descripción
Opcional	Lenguajes	Go, TypeScript, Python
Opcional	Framework	FastAPI, Flask, Python, Express, NestJS, Gin
Opcional	Comunicación	RabbitMQ, Kafka
Opcional	Seguridad y Sesiones	JWT, Session Cookies, OAuth
Opcional	Base de datos	MSSQL, MySQL, PostgreSQL, MongoDB
Obligatorio	Registry	Dockerhub
Obligatorio	Control de Versiones	Github
Obligatorio	Nube	AWS
Opcional	CI/CD	Jenkins, GithubActions
Opcional	Testing	Jest/Mocha, PyTest, go test -coverprofile
Opcional	Documentación, Diseño, Modelado	Excalidraw, Figma, Canva, LucidChart, Draw.io , StarUML, Visual Paradigm, FossFlow

Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	13/06/2026	15/06/2026
Elaboración	13/06/2026	15/06/2026
Calificación	16/06/2026	17/06/2026

Consideraciones

- **Fecha límite de entrega:** 15 de Junio a las 23:59
- **Nombre del repositorio:** SA_PRACTICA_GX
- **Colaborador:** *Samashoas*
- **Medio de entrega:** UEDI
- **Formato de entrega:** Formato Markdown que incluya tabla de integrantes, índice, introducción, desarrollo de todos los diagramas/documentación técnica y conclusiones.
- No se permite el uso de herramientas como supabase o prisma para la gestión de la base de datos (**Si está permitido el uso de otro ORM**)
- Se deben cargar los archivos crudos de la documentación al repositorio, de lo contrario el diagrama no será válido.
- Las construcciones de las imágenes de Docker se debe hacer exclusivamente por medio del CI/CD
- No está permitido bajar la máquina virtual hasta DESPUÉS DE LA CALIFICACIÓN
- **No se permiten commits fuera del horario establecido**
- **No se permiten despliegues fuera del horario establecido**
- **No se permiten accionamientos de CI/CD fuera del horario establecido**
- **SI NO SE HAN CALIFICADO LA PRÁCTICA 3 USEN OTRA VM**